

Response of LTI Discrete Time System in Time Domain

Objective:

The objective of this lab is to introduce students to learn different response of LTI discrete time system in time domain. Specifically, this lab will cover how to implement all four different responses functionality and plot them using Python.

Equipment Needed:

Python 3.x

NumPy library

Matplotlib library

Introduction

To understand the response of a Linear Time-Invariant (LTI) discrete-time system in the time domain:

$$y(n) = - \sum_{m=1}^N a_m y(n-m) + \sum_{m=0}^M b_m x(n-m)$$
$$\underbrace{y(n) + \sum_{m=1}^N a_m y(n-m)}_{\text{L.H.S.}} = \underbrace{\sum_{m=0}^M b_m x(n-m)}_{\text{R.H.S.}}$$

$y[n]$ is the output of the system at time n . $x[n]$ is the input to the system at time n . a_m and b_m is the output, input coefficients of the system. N is the length of the input sequence & M is the length of the output sequence.

Response of an LTI Discrete System can also be written as **the sum of homogenous solution $y_h(n)$ and particular solution $y_p(n)$:**

$$y(n) = y_h(n) + y_p(n)$$

$y_h(n)$: the response of the system when no input is applied, with system's initial conditions.

$y_p(n)$: the response of the system to a particular input signal, ignoring initial conditions.

1. Zero-Input Response (Homogeneous Solution):

The zero-input response, also known as the homogeneous solution, represents the system's response when no input signal is applied. It is solely determined by the initial conditions of the system.

Mathematical Equation:

The zero-input response of an LTI system can be expressed using the system's difference equation and initial conditions. For a system represented by the difference equation:

$$y_{zi}(n) = \sum_0^N a_k y_{zi}(n - k)$$

where **ak** are the system output coefficients and $y(n-k)$ represents the system's output at time $n-k$.

Example:

Consider the difference equation **$y(n)=0.5y(n-1)+0.3y(n-2)$** with initial conditions $y(-1)=1$ and $y(-2)=2$. To find the zero-input response, we compute the system's response without any input signal applied.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

# Define the difference equation coefficients
a1, a2 = 0.5, 0.3

# Define the initial conditions
ic = [1, 2]

# Function for calculating the zero-input response
def zero_input_response(a1, a2, ic):
    """
    Calculate the zero-input response of an LTI system.

    Parameters:
    a1 (float): Coefficient of the difference equation.
    a2 (float): Coefficient of the difference equation.
    ic (list): Initial conditions of the system.

    Returns:
    list: Zero-input response of the LTI system.
    """
    if len(ic) < 2:
        print("At least 2 initial conditions are required for a 2nd order system."
              "Adding a zero-valued initial condition to make the length 2.")
        ic += [0]
    a = [1, -a1, a2]
    zi_response = [sum(ai * ic[m] for m, ai in enumerate(ai_array)) for ai_array in zip(a, ic[::-1])]
```

```

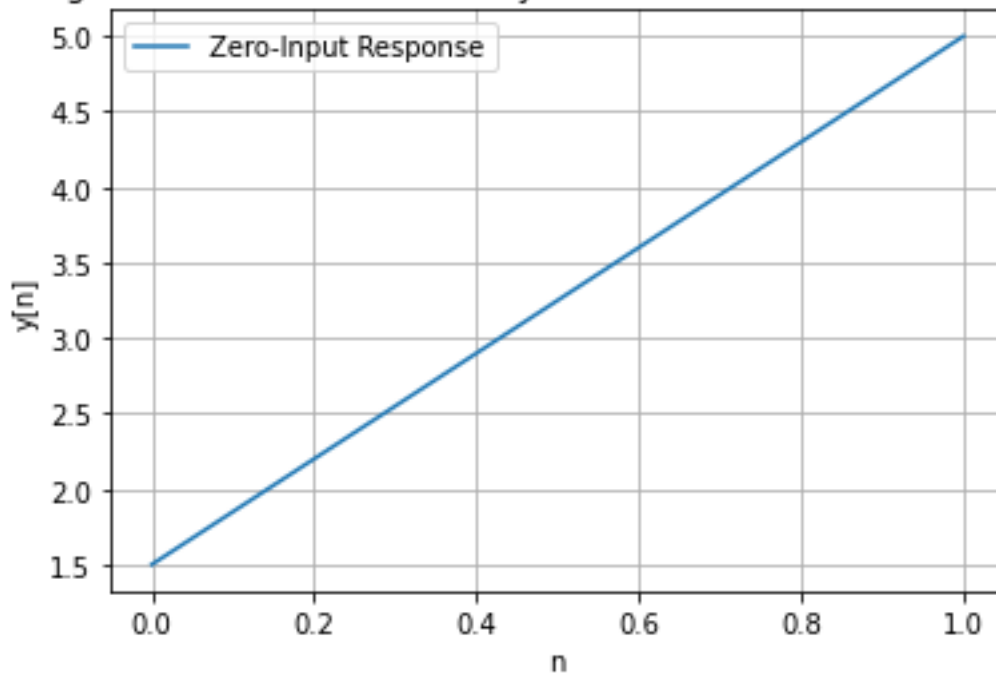
return zi_response[::-1]

# Calculate the homogeneous solution
zi_response = zero_input_response(a1, a2, ic)

# Plot the homogeneous solution
n = np.arange(len(zi_response))
plt.plot(n, zi_response, label='Zero-Input Response')
plt.xlabel('n')
plt.ylabel('y[n]')
plt.title('Homogeneous Solution of an LTI System (2nd Order Difference Equation)')
plt.legend()
plt.grid()
plt.show()

```

Homogeneous Solution of an LTI System (2nd Order Difference Equation)



2. Particular Solution:

The particular solution represents the system's response to a specific input signal, ignoring the initial conditions. It captures the transient behavior of the system caused by the input signal.

Mathematical Equation:

The particular solution of an LTI system can be obtained by simulating the system with the given input signal.

$$y_p(n) = \text{System Simulation with Input Signal } x(n)$$

Example:

For the same difference equation $y(n) = x(n) + 0.3x(n-1) + 0.5x(n-2)$, suppose the input signal is a unit step function $x(n) = u(n)$. We simulate the system with this input signal to find the particular solution.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

# Define the difference equation coefficients
a = [1, -2, -1.6]
b = [0.3, 0.2, 0.1]
n = np.arange(0, 3, 1)

# Input signal (unit step)
x = np.heaviside(n, 1)

def particular_solution(a, b, x):
    """
    Calculate the particular solution of an LTI system.

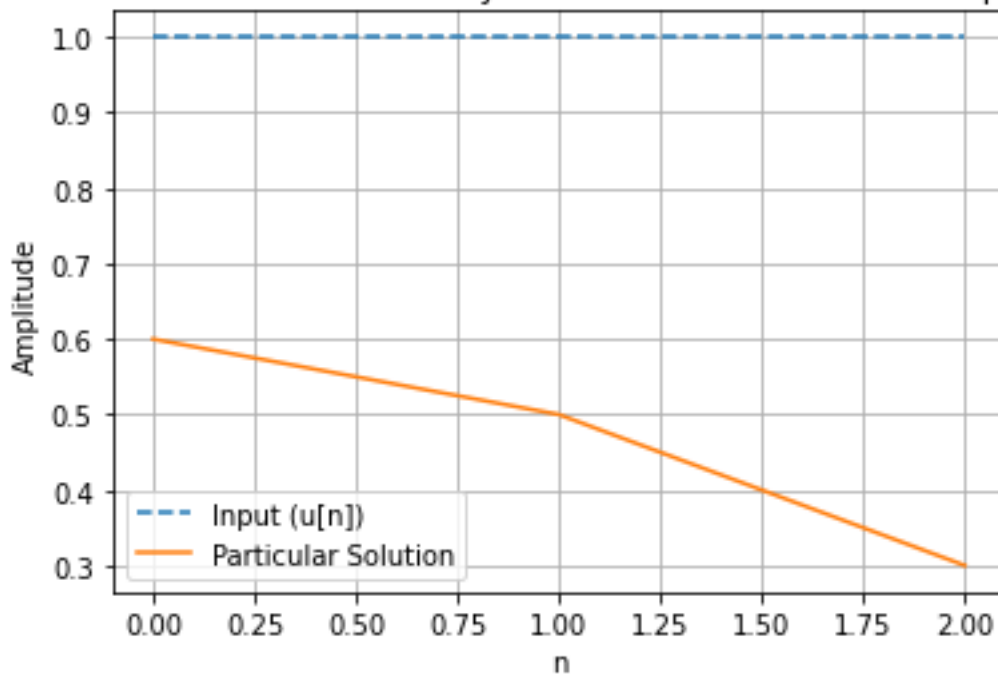
    Parameters:
    a (list): Coefficients of the difference equation (a[0], a[1], ..., a[n]).
    b (list): Coefficients of the difference equation (b[0], b[1], ..., b[m]).
    x (list): Input signal to the LTI system.

    Returns:
    list: Particular solution of the LTI system.
    """
    particular = [sum(xb * bi for xb, bi in zip(x[n:], b)) for n in range(len(x))]
    return particular

# Calculate the particular solution
particular_sol = particular_solution(a, b, x)

# Plot the homogeneous solution
# n = np.arange(len(particular_sol))
plt.plot(x, label="Input (u[n])", linestyle="--")
plt.plot(particular_sol, label="Particular Solution")
plt.xlabel("n")
plt.ylabel("Amplitude")
plt.title("Particular Solution of an LTI System (2nd Order Difference Equation)")
plt.legend()
plt.grid(True)
plt.show()
```

Particular Solution of an LTI System (2nd Order Difference Equation)



3. Zero-State Response:

The zero-state response or forced response is obtained from the sum of homogeneous solution and particular solution and evaluating the constants with zero initial conditions.

Mathematical Equation:

$$y_{zs}(n) = y_h(n) + y_p(n) \quad | \text{ with constants } C_1, C_2, \dots, C_N \text{ evaluated with zero initial conditions}$$

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

def zero_state_response(a, b, x):
    """
    Calculate the zero-state response of an LTI system.

    Parameters:
    a (list): Coefficients of the difference equation (a[0], a[1], ..., a[n]).
    b (list): Coefficients of the difference equation (b[0], b[1], ..., b[m]).
    x (list): Input signal to the LTI system.

    Returns:
    list: Zero-state response of the LTI system.
    """
    # Calculate particular solution
    particular_solution = [sum(xb * bi for xb, bi in zip(x[n:], b)) for n in range(len(x))]
```

```

# Zero homogeneous solution (zero initial conditions)
homogeneous_solution = [0] * len(x)

# Zero-state response
zero_state = [ha + pa for ha, pa in zip(homogeneous_solution, particular_solution)]

return zero_state

# Difference equation coefficients
a = [1, -0.5, -0.3]
b = [0.3, 0.2, 0.1]
n = np.arange(0, 3, 1)

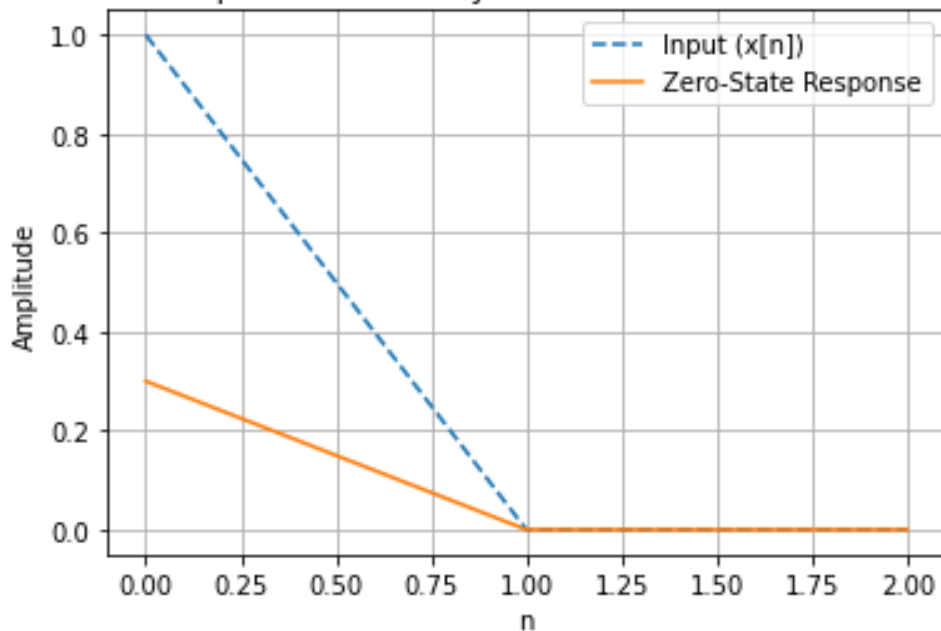
# Input signal (decreasing linear function)
x = np.where(n < 1, 1, 0)

# Calculate zero-state response
zero_state = zero_state_response(a, b, x)

# Plot the input and output
plt.plot(x, label="Input (x[n])", linestyle="--")
plt.plot(zero_state, label="Zero-State Response")
plt.xlabel("n")
plt.ylabel("Amplitude")
plt.legend()
plt.grid(True)
plt.show()

```

Zero State Response of an LTI System (2nd Order Difference Equation)



4. Total Response:

The overall response of the system to a given input signal, considering both the homogeneous solution and the particular solution. This response can be obtained by adding the zero-input response (homogeneous solution) and the zero-state response (particular solution).

The total response is given by sum of zero-input response and zero-state response.

Mathematical Equation:

$$y(n) = y_{zi}(n) + y_{zs}(n)$$

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

def zero_input_response(a, ic):
    """
    Calculate the zero-input response of an LTI system (homogeneous solution).

    Parameters:
    a (list): Coefficients of the difference equation (a[0], a[1], ..., a[n]).
    ic (list): Initial conditions of the system.

    Returns:
    list: Zero-input response of the LTI system.
    """
    zi_response = [sum(ai * ic[m] for m, ai in enumerate(a[1:], 1)) for n in range(len(ic))]
    return zi_response[:-1]

def zero_state_response(a, b, x):
    """
    Calculate the zero-state response of an LTI system (particular solution with zero initial conditions).

    Parameters:
    a (list): Coefficients of the difference equation (a[0], a[1], ..., a[n]).
    b (list): Coefficients of the difference equation (b[0], b[1], ..., b[m]).
    x (list): Input signal to the LTI system.

    Returns:
    list: Zero-state response of the LTI system.
    """
    zs_response = [sum(xb * bi for xb, bi in zip(x[n:], b)) for n in range(len(x))]
    return zs_response

def total_response(a, b, x, ic):
    """
    Calculate the total response of an LTI system.
```

Parameters:

a (list): Coefficients of the difference equation ($a[0]$, $a[1]$, ..., $a[n]$).

b (list): Coefficients of the difference equation ($b[0]$, $b[1]$, ..., $b[m]$).

x (list): Input signal to the LTI system.

ic (list): Initial conditions of the system.

Returns:

list: Total response of the LTI system.

"""

```
zero_state = zero_state_response(a, b, x)
```

```
zero_input = zero_input_response(a, ic)
```

```
return [zs + zi for zs, zi in zip(zero_state, zero_input)]
```

```
# Difference equation coefficients
```

```
a = [1, -0.5, -0.3]
```

```
b = [0.3, 0.2, 0.1]
```

```
# Initial conditions
```

```
ic = [0, 1, -2]
```

```
# Input signal (decreasing linear function)
```

```
n = np.arange(0, 3, 1)
```

```
x = np.where(n >= 0, 1, 0)
```

```
# Calculate total response
```

```
total_response_output = total_response(a, b, x, ic)
```

```
# Plot the input, zero-input response, zero-state response, and total response
```

```
fig, axs = plt.subplots(1, 2, figsize=(8, 12))
```

```
# Input signal
```

```
axs[0].stem(n, x, label="Input (x[n])")
```

```
axs[0].set_xlabel("n")
```

```
axs[0].set_ylabel("Amplitude")
```

```
axs[0].set_title("Linearly Decreasing Function")
```

```
axs[0].grid(True)
```

```
# Zero-input response (homogeneous solution)
```

```
axs[1].stem(n, total_response_output, label="Total Response")
```

```
axs[1].set_xlabel("n")
```

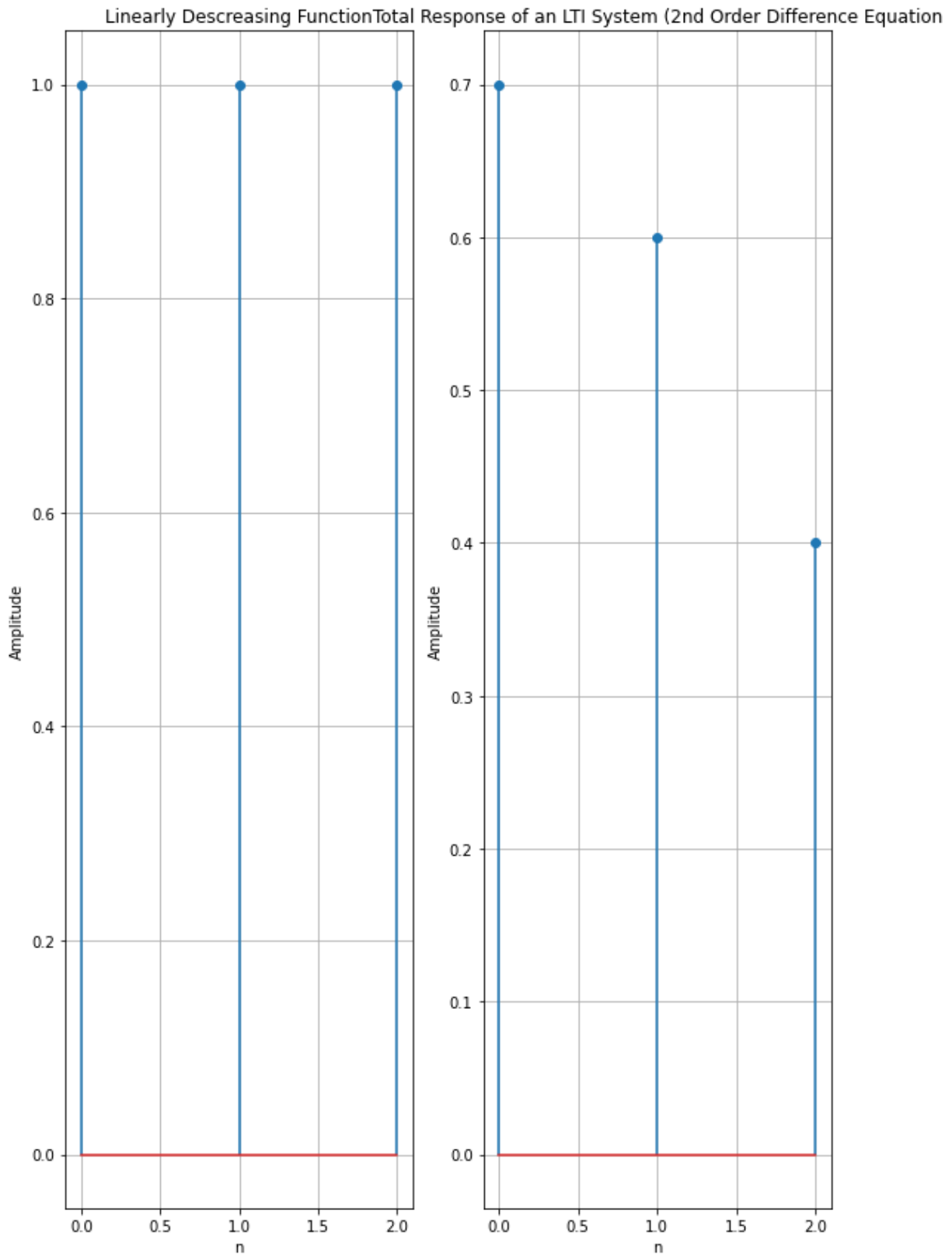
```
axs[1].set_ylabel("Amplitude")
```

```
axs[1].set_title("Total Response of an LTI System (2nd Order Difference Equation)")
```

```
axs[1].grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```



LAB TASKS

- Write Code for all four responses of first order discrete time system governed by the difference equation, $y(n) = -0.8 y(n - 1) + x(n) + 3 x(n - 1)$.
When the input is unit step, and with initial condition a) $y(-1) = 0$ b) $y(-1) = 2.5$.
- Determine the response $y(n)$, $n > 0$ of the system described by the second order difference equation, $y(n) - 0.2 y(n - 1) - 0.03 y(n - 2) = x(n) + 0.4 x(n - 1)$.
When the input signal is, $x(n) = 0.2n u(n)$ and with initial conditions $y(-2) = 0$, $y(-1) = 0.5$.